Contest

template

a86a27

```cpp
#include <bits/stdc++.h>
using namespace std;

#define rep(i, a, b) for (int i = (a); i < (b); ++i)
#define all(x) begin(x), end(x)
#define sz(x) (int)(x).size()
typedef long long ll;
typedef pair<int, int> pii;
typedef vector<int> vi;

int main() {
  cin.tie(0)->sync_with_stdio(0);
  cin.exceptions(cin.failbit);
}
```

hash

cd55fe

```cpp
# Hash de un archivo para comparar con el notebook: bash hash.sh < archivo.h
tr -d '[:space:]' | md5sum | cut -c-6
```

Estructuras de datos

DSU

484c87

Desc: Disjoint Set Union con union by size y path compression. join devuelve false si ya estaban unidos.

Tiempo: $O(\alpha(N))$ amortizado

```cpp
struct DSU {
  vi p;
  DSU(int n) : p(n, -1) {}
  int find(int x) { return p[x] < 0 ? x : p[x] = find(p[x]); }
  bool same(int a, int b) { return find(a) == find(b); }
  int size(int x) { return -p[find(x)]; }
  bool join(int a, int b) {
    a = find(a), b = find(b);
    if (a == b) return false;
    if (p[a] > p[b]) swap(a, b);
    p[a] += p[b];
    p[b] = a;
    return true;
  }
};
```

SegmentTree

db129c

Desc: Segment tree iterativo. Actualización puntual y consulta en [l, r). op debe ser asociativa con neutro e.

Uso: SegTree st(n); st.update(i, v); st.query(l, r);

Tiempo: $O(\log N)$ por operación

```cpp
struct SegTree {
  typedef ll T;
  static constexpr T e = LLONG_MIN;
  T op(T a, T b) { return max(a, b); }
  int n;
  vector<T> t;
  SegTree(int n = 0) : n(n), t(2 * n, e) {}
  void update(int i, T v) {
    for (t[i += n] = v; i >>= 1;) t[i] = op(t[2 * i], t[2 * i + 1]);
  }
  T query(int l, int r) {
    T a = e, b = e;
    for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
      if (l & 1) a = op(a, t[l++]);
      if (r & 1) b = op(t[--r], b);
    }
    return op(a, b);
  }
};
```

Grafos

Dijkstra

b41be9

Desc: Distancias mínimas desde s con pesos no negativos. g[u] = lista de (v, w). Inalcanzable = LLONG_MAX.

Tiempo: $O((V + E) \log V)$

```cpp
vector<ll> dijkstra(const vector<vector<pair<int, ll>>>& g, int s) {
  vector<ll> d(sz(g), LLONG_MAX);
  priority_queue<pair<ll, int>, vector<pair<ll, int>>, greater<>> pq;
  pq.push({d[s] = 0, s});
  while (!pq.empty()) {
    auto [du, u] = pq.top();
    pq.pop();
    if (du != d[u]) continue;
    for (auto [v, w] : g[u])
      if (du + w < d[v]) pq.push({d[v] = du + w, v});
  }
  return d;
}
```

Teoría de números

ModInt

1c8ba9

Desc: Aritmética modular con MOD primo. inv usa el pequeño teorema de Fermat.

Tiempo: $O(1)$ por operación, $O(\log MOD)$ para pow e inv

```cpp
const ll MOD = 1e9 + 7;
struct Mint {
  ll x;
  Mint(ll v = 0) : x(v % MOD) { if (x < 0) x += MOD; }
  Mint operator+(Mint o) const { return x + o.x; }
  Mint operator-(Mint o) const { return x - o.x; }
  Mint operator*(Mint o) const { return x * o.x; }
  Mint pow(ll e) const {
    Mint r = 1, b = *this;
    for (; e; e >>= 1, b = b * b)
      if (e & 1) r = r * b;
    return r;
  }
  Mint inv() const { return pow(MOD - 2); }
  Mint operator/(Mint o) const { return *this * o.inv(); }
  bool operator==(Mint o) const { return x == o.x; }
};
```

Strings

PrefixFunction

077231

Desc: pi[i] = longitud del mayor borde propio de s[0..i]. match devuelve las posiciones donde empieza p en s.

Tiempo: $O(N + M)$

```cpp
vi prefixFunction(const string& s) {
  vi pi(sz(s));
  rep(i, 1, sz(s)) {
    int k = pi[i - 1];
    while (k && s[i] != s[k]) k = pi[k - 1];
    pi[i] = k + (s[i] == s[k]);
  }
  return pi;
}

vi match(const string& s, const string& p) {
  vi pi = prefixFunction(p + '\0' + s), res;
  rep(i, sz(p) + 1, sz(pi))
    if (pi[i] == sz(p)) res.push_back(i - 2 * sz(p));
  return res;
}
```